SAVEETHA SCHOOL OF ENGINEERING

SIMATS Engineering

Department of Computer Science and Engineering

OPERATING SYSTEMS

ASSIGNMENT 3

CO3: Evaluate memory-management mechanisms such as linking, paging and segmentation to determine their effectiveness for different system requirements – linking and loading, paging, segmentation, virtual memory, demand paging, page-replacement techniques and disk management.

(Total Marks – 50)

Name	B.Monish
Register Number	1925260207
Course Code	CSA0403
Course Name	Operating Systems
Assignment	Assignment 3 – CO3

Test system for all experiments: Ubuntu 24.04 LTS (Linux), gcc 13.3.0, page size 4096 bytes (getconf PAGESIZE), 1 CPU core, about 4 GB of RAM, no swap. Programs were compiled with gcc -std=c11 -Wall -Wextra (with -pthread or -ldl where needed).

Q1 – Linking, Loading and Address Binding

Q1 (K5 – Evaluate) **[6 Marks]**

Context: A bank's branch software is built from many modules and libraries. After a security fix to the encryption library, every branch must receive a new 200 MB executable.

Implementation: build with static linking and with shared libraries; compare sizes and list libraries with ldd; load a library with dlopen()/dlsym(); show the load addresses (/proc/self/maps).

Evaluation: compile-time, load-time and execution-time binding; linker and loader; static vs dynamic linking (memory, start-up, updating); position-independent code.

Experiments: start-up time and memory; replace the shared library without recompiling; two copies – which pages are shared; library missing.

Conclusion: which linking method should the bank use?

Alternative Concept: load the encryption library with dlopen() only when needed – start-up time, memory when encryption is not needed, error handling.

Answer:

1.1 Programs used

A small "bank" program calls an encryption library (crypto.c) that has two functions: crypto_version() and encrypt_buf() (XOR with a key).

```
/* crypto.c */
#include "crypto.h"
const char *crypto_version(void) { return "encryption library v1 (old algorithm)";
}
void encrypt_buf(char *buf, int n, char key) { for (int i = 0; i < n; i++) buf[i]
^= key; }

/* bank.c */
int main(void) {
    char msg[] = "ACCOUNT 192511355 BALANCE 5000";
    printf("%s\n", crypto_version());
    encrypt_buf(msg, strlen(msg), 0x5A); encrypt_buf(msg, strlen(msg), 0x5A);   /*
encrypt, then decrypt */
    printf("round trip ok: %s\n", msg); }
```

Commands used:

```
gcc -c -fPIC crypto.c -o crypto_pic.o && gcc -shared -o libcrypto_bank.so
crypto_pic.o
gcc -std=c11 -Wall -Wextra bank.c -L. -lcrypto_bank -Wl,-rpath,'$ORIGIN' -o
bank_dyn   # dynamic
gcc -std=c11 -Wall -Wextra -static bank.c crypto.c -o bank_static
# static
```

1.2 Size of the executables and ldd

File	Size (bytes)	Remark
bank_static	785,560	Contains the program, the encryption code and all the C-library code that it uses
bank_dyn	16,168	Only the program; the libraries are separate files
libcrypto_bank.so	15,224	Shared encryption library (updated on its own)

The static executable is about **49 times larger** (785,560 / 16,168). The command **ldd bank_dyn** lists the shared libraries that the loader will map:

```
linux-vdso.so.1 (0x00007efd73430000)
libcrypto_bank.so => /home/claude/a3/./libcrypto_bank.so (0x00007efd7341e000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007efd73000000)
/lib64/ld-linux-x86-64.so.2 (0x00007efd73432000)
```

ldd bank_static answers “not a dynamic executable”, i.e. it needs no libraries. Both programs print the same output: “encryption library v1 (old algorithm)” and “round trip ok: ACCOUNT 192511355 BALANCE 5000”.

1.3 Loading a library at run time with dlopen() and dlsym()

```
void *h = dlopen("./libcrypto_bank.so", RTLD_NOW);
if (!h) { printf("encryption not available: %s\n", dlerror()); return 1; }
const char *(*ver)(void) = (const char *(*)(void))dlsym(h, "crypto_version");
void (*enc)(char *, int, char) = (void (*)(char *, int, char))dlsym(h,
"encrypt_buf");
if (!ver || !enc) { printf("symbol missing: %s\n", dlerror()); return 1; }
printf("loaded at run time: %s\n", ver());   enc(msg, n, 0x5A);   enc(msg, n,
0x5A);
dlclose(h);
```

Compile: **gcc -std=gnu11 -Wall -Wextra dl.c -ldl -o bank_dlopen**. Output: “loaded at run time: encryption library v1 (old algorithm)” and “round trip ok: ACCOUNT 192511355 BALANCE 5000”. dlopen() asks the loader to map the library, dlsym() returns the address of a function by name and the program calls it through a function pointer.

1.4 Addresses at which the program and its libraries are loaded (/proc/self/maps)

The program reads /proc/self/maps and prints the lines of itself, the heap, libc, the encryption library and the stack (only address range, permissions and file are shown; the remarks in brackets were added by me):

```
5580d4eb9000-5580d4eba000 r--p /home/claude/a3/maps_dyn             (ELF header,
read-only)
5580d4eba000-5580d4ebb000 r-xp /home/claude/a3/maps_dyn             (code)
5580d4ebd000-5580d4ebe000 rw-p /home/claude/a3/maps_dyn             (data)
5580e2d51000-5580e2d72000 rw-p [heap]
7f2cdbc6a000-7f2cdbc6b000 r--p /home/claude/a3/libcrypto_bank.so
7f2cdbc6b000-7f2cdbc6c000 r-xp /home/claude/a3/libcrypto_bank.so    (library code,
executable)
7f2cdbc6e000-7f2cdbc6f000 rw-p /home/claude/a3/libcrypto_bank.so    (library data)
7ffc161d2000-7ffc161f3000 rw-p [stack]
```

Observations: the code is mapped r-xp (read and execute, not writable) and the data rw-p (not executable); the libraries are in the high 0x7f… area, the program at 0x55…, the stack at 0x7ffc…. In a second run all the addresses were different (for example the heap at 55e7d433b000 instead of 5580e2d51000), which shows **address-space layout randomisation (ASLR)** – the code works at any address because it is position independent.

1.5 Address binding, linker and loader, position-independent code

Binding time	Meaning	Example / requirement
Compile time	The compiler already knows where the program will be in memory and generates absolute addresses; to move it, it must be recompiled	MS-DOS .COM files; embedded systems with fixed memory

Binding time	Meaning	Example / requirement
Load time	The compiler generates relocatable code; the loader adds the starting address when loading; to move it, the program is reloaded	Older executables with relocation tables
Execution time	The binding is delayed until run time; the process can be moved during execution; needs hardware (base/limit registers or an MMU with paging)	Modern OS with virtual memory – used by Linux and Windows

- **Linker:** combines the object files (.o) and libraries into one executable, resolves the symbols (a call to encrypt_buf is connected to its definition) and relocates the sections. With dynamic linking it only records which libraries and symbols are needed.
- **Loader:** a part of the OS (and the dynamic loader ld.so) that reads the executable from disk, creates the process address space, maps the code and data, loads the shared libraries, performs the relocations, and starts the program at its entry point.
- **Position-independent code (PIC):** code that works wherever it is loaded because it uses relative addressing (for example RIP-relative addressing) and an indirection table (GOT/PLT) for the addresses of external functions and global data. A shared library is compiled with -fPIC so that one copy of its code in memory can be mapped at a different address in each process and shared by all of them without changes.

1.6 Evaluation of static and dynamic linking

Criterion	Static linking	Dynamic linking
Memory use	Every program has its own copy of the library code (bank_static 785 KB; 100 programs = 100 copies)	One copy of the library in RAM is shared by all processes (libc code was shared by several processes, see 1.8)
Start-up time	Faster – no libraries to find and map (measured 777 and 825 µs per run)	Slower – the dynamic loader opens, maps and relocates the libraries (measured 1020 and 964 µs per run)
Ease of updating	Hard – every program must be re-linked and sent again (the bank's 200 MB executable for every fix)	Easy – only the small library file is replaced and all programs use the fix at the next start
Dependency risk	None – self-contained	The library must exist and have a compatible version ("dependency hell")

1.7 Experiments

(a) Start-up time and memory. Each program was started 500 times in a loop (two series): the static version took 777 µs and 825 µs per run, the dynamic version 1020 µs and 964 µs; so the dynamic version is about 150–250 µs slower to start (the cost of loading libc and the library). Memory of one running process (VmSize / VmRSS from /proc/self/status): static **1012 kB / 724 kB**; dynamic **2708 kB / 1400 kB**. A single process therefore uses more memory with dynamic linking, because the libc and the loader are mapped completely (the pages are only touched when used); but the shared pages are counted in each process's RSS even though they exist once in RAM (see 1.8), so with many programs dynamic linking saves memory.

(b) Replacing the library without recompiling. The library source was changed to report "v2 (SECURITY FIX applied)", rebuilt into the same file name libcrypto_bank.so, and the **unchanged** programs were run:

```
$ ./bank_dyn
encryption library v2 (SECURITY FIX applied)       <- new behaviour, same
executable (still 16,168 bytes)
$ ./bank_static
encryption library v1 (old algorithm)                <- still the old code
```

The dynamic program picked up the fix immediately, whereas the static program kept the old code until it is rebuilt – this is exactly the bank's problem: with dynamic linking a branch needs only a 15 KB library file instead of a new 200 MB executable.

(c) Pages shared by two copies. Two copies of the dynamic program were run at the same time and /proc/<pid>/smaps was read (Rss = pages in RAM, Shared_Clean = pages shared with other processes, Pss = proportional share):

Mapping (same in both copies)	Rss	Shared_Clean	Pss (each copy)
libc.so.6	1072 kB	1048 kB	262 kB
libcrypto_bank.so	16 kB	–	12 kB

For libc, 1048 of the 1072 kB are shared clean pages: the pages exist once in physical memory and are mapped in both processes (and in other running programs); the Pss of 262 kB shows that the cost is divided among the processes that use it (about four processes were using libc). So the code pages of a library are shared, while the data pages (rw-p) are private (copy-on-write).

(d) Library missing. The library file was moved away and the programs were run: the dynamic program stopped before main() with the loader message **"error while loading shared libraries: libcrypto_bank.so: cannot open shared object file: No such file or directory"**; the static program is not affected. The dlopen program printed: **"encryption not available: ./libcrypto_bank.so: cannot open shared object file: No such file or directory"** and could handle the error by itself.

1.8 Conclusion – which method should the bank use?

The bank should use **dynamic (shared) linking** for the encryption library (and the other libraries that change often). A security fix then needs only a small library file (15 KB in my test, not 200 MB) to be distributed to the branches; no program must be rebuilt; the library code is shared in memory by all programs; and the start-up cost is only a fraction of a millisecond. The risks – a missing or incompatible library – are handled by installing the library with the application package, checking versions (library name with version number, for example libcrypto.so.3), and keeping a copy of the previous version for roll-back. Static linking would be used only for a small, rarely updated tool that must run on a machine without the library (for example a recovery tool).

1.9 Alternative concept – dlopen() only when needed

Criterion	Linked dynamically at start-up	dlopen() when needed
Start-up time	The loader loads the library before main() (about 150–250 µs more than static here)	No library work at start-up; the cost (a few tens to hundreds of µs) is paid at the first use
Memory when encryption is not needed	The library is mapped (and its pages are loaded when used)	Nothing is mapped; no memory is used at all until dlopen()
Error handling	The program does not start at all if the library is missing	The program starts; dlopen()/dlsym() return NULL and the program must test them and use dlerror() – it can continue without encryption or show a message
Complexity of the code	Normal function calls	Function pointers, casts and explicit error checks; the compiler does not check the function types

Run-time loading is suitable for optional features and plug-ins; for the main encryption of a bank (which is always required) normal dynamic linking is simpler and safer, as errors are found at start-up instead of in the middle of a transaction.

Q2 – Paging

Q2 (K5 – Evaluate) **[8 Marks]**

Context: A multi-user college server; contiguous allocation left small unusable gaps and large programs could not be loaded though enough memory was free.

Implementation: external fragmentation diagram; logical → physical translation; page-table size for 32-bit and 48-bit addresses with 4 KB pages; effective access time with and without TLB.

Evaluation: how paging removes external fragmentation but keeps internal fragmentation; page sizes 1 KB, 4 KB, 2 MB; hierarchical, hashed and inverted page tables; protection and sharing.

Experiments: internal fragmentation of 100 random processes; EAT for hit ratios 80/95/99%; getconf PAGESIZE and /usr/bin/time -v; test addresses at page start, end and beyond the last page.

Conclusion: page size for the college server.

Complete the Code: TODO 1; change PAGE_SIZE to 2048 and calculate by hand first.

Alternative Concept: inverted page table instead of a multi-level table.

Answer:

2.1 External fragmentation in contiguous allocation



In contiguous allocation every program needs one unbroken block. After programs finish and others start, free memory is broken into small holes (here 8, 6, 7 and 8 units). The total free memory (29 units) is large, but no single hole can hold a 20-unit program – this is **external fragmentation**. Compaction (moving programs together) would solve it but is slow and needs run-time binding.

2.2 Address translation with a page table

The logical address is divided into the page number p = address / page size and the offset d = address mod page size. The page table gives the frame number f for page p, and the physical address = f × page size + d. The offset is the same in the page and in the frame.

2.3 Size of the page table (4 KB pages)

Address space	Number of pages = 2^(bits – 12)	Entry size	Size of a single-level page table (per process)
32-bit	2^20 = 1,048,576	4 bytes	4 MB
48-bit	2^36 = 68,719,476,736	8 bytes	512 GB (!)

A 512 GB table for each process is impossible, which is why 64-bit systems use hierarchical page tables: the table is split in levels and the parts for unused addresses are not created. x86-64 Linux uses four levels of 512 entries (9 bits each); one table fits exactly into a 4 KB page (512 × 8 bytes), and 4 × 9 + 12 = 48 bits.

2.4 Effective access time (EAT) with and without a TLB

Assumptions: one memory access = 100 ns, TLB look-up = 20 ns, single-level page table. **Without a TLB** every reference needs two memory accesses (page table + data): EAT = 2 × 100 = **200 ns**. **With a TLB**: EAT = h × (20 + 100) + (1 – h) × (20 + 100 + 100).

TLB hit ratio h	Calculation	EAT
80%	0.80 × 120 + 0.20 × 220	140 ns (40% slower than 100 ns)
95%	0.95 × 120 + 0.05 × 220	125 ns (25% slower)
99%	0.99 × 120 + 0.01 × 220	121 ns (21% slower)

Even with a 99% hit ratio the access is almost twice as fast as without a TLB (121 ns against 200 ns); a hit ratio below 90% would waste much of the benefit, which is why TLB reach (number of entries × page size) matters.

2.5 Paging and fragmentation

Paging divides the logical memory into fixed-size pages and the physical memory into frames of the same size; a process can be placed into **any** free frames, which need not be next to each other, so **no external fragmentation** occurs – every free frame can be used. **Internal fragmentation** remains because the last page of a process is usually only partly used: on average half a page is wasted per process (for example a 72,766-byte process needs 18 pages of 4,096 bytes = 73,728 bytes, so 962 bytes are wasted).

2.6 Page sizes of 1 KB, 4 KB and 2 MB – experiment

Program q2f.c creates 100 processes with random sizes between 10 KB and 50 MB (total 2,706.9 MB) and calculates the number of pages and the wasted memory for each page size:

Page size	Pages (= page-table entries)	Internal fragmentation (total)	Per process	% of memory
1 KB	2,771,920	49.8 KB	0.5 KB	0.0018%
4 KB	693,022	217.8 KB	2.2 KB	0.0079%

Page size	Pages (= page-table entries)	Internal fragmentation (total)	Per process	% of memory
2 MB	1,403	101,473.8 KB (about 99 MB)	1,014.7 KB	3.66%

The result agrees with the theory (about half a page per process: 0.5 KB, 2 KB and 1 MB). The **small page** wastes little memory but needs a very large page table (2.77 million entries) and more TLB misses; the **huge page** needs a tiny table (1,403 entries) and gives a much better TLB reach, but wastes about 1 MB per process, which is bad for many small programs and also makes disk transfers of a page large. **4 KB** is a compromise.

2.7 Hierarchical, hashed and inverted page tables

Type	How it works	Good / bad
Hierarchical (multi-level)	The page number is split; the outer table points to inner tables; inner tables for unused regions are not allocated (x86-64: four levels)	Saves space for sparse address spaces; a miss needs one memory access per level (4 for x86-64)
Hashed	The page number is hashed to a table of chains; each element holds (page, frame, next); used for address spaces above 32 bits	Fast average look-up (O(1)); needs chain search on collisions
Inverted	One table for the whole system with one entry per physical frame (process ID, page); the entry found gives the frame	Table size depends on RAM, not on the number of processes; the search is slow, so a hash is used; sharing pages is difficult

2.8 Protection and shared pages

- **Protection:** every page-table entry has bits: valid/invalid (is the page part of the process?), read/write/execute permission, and user/supervisor. The MMU checks them on every access; a violation causes a trap to the OS. An address beyond the last page is trapped by the valid bit (or the page-table length register).
- **Shared pages:** reentrant code (for example the editor or compiler) is kept as one copy in physical memory; the page-table entries of different processes point to the same frames. The code pages are read-only; the data pages are private to each process.

2.9 Real system measurements

getconf PAGESIZE → 4096. A program was run that allocates and writes 100 MB (/usr/bin/time -v ./pf):

```
touching 100 MB: minor page faults=25604 (100 MB / 4 KB = 25600 pages), major
faults=0
Maximum resident set size (kbytes): 103956
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 25675
```

100 MB / 4 KB = 25,600 pages, and the program had 25,604 minor faults (the other faults are for the start-up of the program): each page was loaded into a frame the first time it was written (demand paging). There were no major faults, because no page had to be read from the disk.

2.10 Testing addresses at the start and end of a page and beyond the last page

Page size 4096, a page table of 4 pages {page 0 → frame 9, page 1 → frame 3, page 2 → not in memory, page 3 → frame 6}:

```
     0 -> page 0 offset    0 : physical 36864      (first byte of page 0 = frame 9
× 4096)
  4095 -> page 0 offset 4095 : physical 40959      (last byte of page 0)
  4096 -> page 1 offset    0 : physical 12288      (first byte of page 1 – the
next page uses another frame)
  8191 -> page 1 offset 4095 : physical 16383
  8192 -> page 2 offset    0 : page fault
16383 -> page 3 offset 4095 : physical 28671      (last valid byte)
16384 -> page 4 offset    0 : invalid (beyond last page)
```

The addresses 4095 and 4096 are next to each other in the logical memory, but their physical addresses (40,959 and 12,288) are far apart – the pages are in different frames. The address 16,384 is outside the process and is trapped.

2.11 Completed code (TODO 1) and output

```
#include <stdio.h>

#ifndef PAGE_SIZE
#define PAGE_SIZE 1024
#endif
#define PAGES 8

/* Page table of a process: frame number, or -1 if the page is not in memory. */
int page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};

int main(void)
{
    unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049};
    for (int i = 0; i < 6; i++) {
        unsigned a = logical[i];
        unsigned p = a / PAGE_SIZE, d = a % PAGE_SIZE;
        printf("%6u -> page %u offset %6u : ", a, p, d);
        if (p >= PAGES) printf("invalid address\n");
        else if (page_table[p] == -1) printf("page fault\n");
        else printf("physical %u\n", page_table[p] * PAGE_SIZE + d);
    }
    return 0;
}
```

Output (same as the expected output):

```
     0 -> page 0 offset      0 : physical 5120
  1500 -> page 1 offset    476 : physical 2524
  3100 -> page 3 offset     28 : physical 7196
  6200 -> page 6 offset     56 : physical 3128
  9000 -> page 8 offset    808 : invalid address
  2049 -> page 2 offset      1 : page fault
```

Hand calculation with PAGE_SIZE 2048 (page table {5, 2, –1, 7, 0, –1, 3, –1}, 8 pages):

Address	Page = a / 2048, offset = a mod 2048	Frame	Result
0	page 0, offset 0	5	5 × 2048 + 0 = 10240
1500	page 0, offset 1500	5	10240 + 1500 = 11740
3100	page 1, offset 1052	2	4096 + 1052 = 5148
6200	page 3, offset 56	7	14336 + 56 = 14392
9000	page 4, offset 808	0	0 + 808 = 808

Address	Page = a / 2048, offset = a mod 2048	Frame	Result
2049	page 1, offset 1	2	4096 + 1 = 4097

The program (compiled with -DPAGE_SIZE=2048) gave exactly these values: 10240, 11740, 5148, 14392, 808 and 4097. With larger pages, the same page table covers a larger address space (16 KB instead of 8 KB), so the addresses 9000 (earlier invalid) and 2049 (earlier a page fault) are now valid.

2.12 Conclusion – page size for the college server

I would choose the **standard 4 KB page** for the college server. It wastes only about 2 KB per process (0.008% in my experiment) – important for hundreds of small student programs – and the page tables remain reasonable with multi-level tables and a TLB. 1 KB pages would produce a table four times larger and more TLB misses; 2 MB pages would waste about 1 MB for every process (3.7%). Huge pages (2 MB) can be used for a few large programs (for example a database or the web server cache) if they are really needed.

2.13 Alternative concept – inverted page table

With an inverted page table the system keeps one table of F entries (F = number of frames) containing (process ID, page number). To translate (pid, p), the table is searched using a hash of (pid, p) to find the entry; the index of the entry is the frame number.

```
frame = hash_lookup(pid, p);          /* O(1) average, O(chain) worst */
if (frame == NOT_FOUND) page_fault();
physical = frame * PAGE_SIZE + offset;
```

Criterion	Multi-level page table (per process)	Inverted page table
Space	Proportional to the used part of each process's address space: O(processes × used pages), with many processes it grows	Proportional to the physical memory: O(F). For 8 GB RAM and 4 KB frames: 2,097,152 entries × 8–16 B = 16–32 MB, whatever the number of processes
Translation time	Walk of 4 levels: 4 memory accesses on a TLB miss (O(levels))	One hash look-up: about 1–2 accesses on average, but a chain search on collisions (O(1) average, O(F) worst case)
Shared pages	Easy – several tables point to the same frame	Difficult – one entry per frame holds only one (pid, page)

For a server with hundreds of processes the inverted table saves memory, but multi-level tables are simpler and allow sharing, so they are used by most systems; the inverted table is used on some architectures (PowerPC, UltraSPARC).

Q3 – Segmentation

Q3 (K5 – Evaluate) **[6 Marks]**

Context: Student programs have code (read-only, shared), a growing stack and data of different sizes; paging alone treats all alike.

Implementation: translate (segment, offset) with a segment table (base, limit); show what happens when the offset exceeds the limit; draw segmentation with paging (Intel x86); show the segments of a running Linux program (/proc/<pid>/maps).

Evaluation: segmentation and the programmer's view; protection and sharing; external fragmentation; compare pure paging, pure segmentation, segmented paging.

Experiments: translate 8 addresses by hand and verify with a program; trigger a segmentation fault and explain the OS action; fragmentation after loading and removing 20 random segments; table sizes of each scheme.

Conclusion: why modern OSes rely mainly on paging.

Alternative Concept: pure paging instead of segmentation.

Answer:

3.1 Segment table and address translation

A logical address is a pair (segment number s, offset d). The segment table has a **base** (starting physical address) and a **limit** (length) for each segment. If d < limit, the physical address = base + d; otherwise the hardware traps to the OS (addressing error).

Segment	Name	Base	Limit
0	code	1400	1000
1	data	6300	400
2	stack	4300	1100
3	heap	3200	500

3.2 Eight addresses by hand, verified by a program

(s, d)	Hand calculation	Result
(0, 53)	53 < 1000 → 1400 + 53	1453
(0, 1000)	1000 is not < 1000	trap – offset exceeds the limit
(1, 0)	0 < 400 → 6300 + 0	6300
(1, 399)	399 < 400 → 6300 + 399	6699
(2, 1099)	1099 < 1100 → 4300 + 1099	5399
(2, 1100)	1100 is not < 1100	trap – offset exceeds the limit
(3, 499)	499 < 500 → 3200 + 499	3699
(3, 600)	600 is not < 500	trap – offset exceeds the limit

```
#include <stdio.h>
/* segment table: base and limit */
struct seg { const char *name; unsigned base, limit; };
struct seg st[] = {{"code", 1400, 1000}, {"data", 6300, 400}, {"stack", 4300,
1100}, {"heap", 3200, 500}};
int main(void)
{
    unsigned s[] = {0, 0, 1, 1, 2, 2, 3, 3, 4};
    unsigned o[] = {53, 1000, 0, 399, 1099, 1100, 499, 600, 10};
    for (int i = 0; i < 9; i++) {
        printf("(%u, %4u) : ", s[i], o[i]);
        if (s[i] >= 4) { printf("invalid segment number -> trap\n"); continue; }
        if (o[i] >= st[s[i]].limit)
            printf("offset >= limit (%u) in %s -> trap (segmentation fault)\n",
st[s[i]].limit, st[s[i]].name);
        else
            printf("%-5s physical %u + %u = %u\n", st[s[i]].name, st[s[i]].base,
o[i], st[s[i]].base + o[i]);
    }
    return 0;
```

```
}
```

The program output matched the hand calculation for all the eight addresses (and also trapped the invalid segment number (4, 10)):

```
(0,   53) : code  physical 1400 + 53 = 1453
(0, 1000) : offset >= limit (1000) in code -> trap (segmentation fault)
(1,    0) : data  physical 6300 + 0 = 6300
(1,  399) : data  physical 6300 + 399 = 6699
(2, 1099) : stack physical 4300 + 1099 = 5399
(2, 1100) : offset >= limit (1100) in stack -> trap (segmentation fault)
(3,  499) : heap  physical 3200 + 499 = 3699
(3,  600) : offset >= limit (500) in heap -> trap (segmentation fault)
(4,   10) : invalid segment number -> trap
```

When the offset exceeds the limit, the hardware does not access memory; it raises an exception (trap) to the OS, which normally terminates the process with a segmentation fault.

3.3 Triggering a segmentation fault

A program that writes to the address 0 (a null pointer): int p = 0; (write through the null pointer p). Result:

```
about to write to address 0
Segmentation fault
exit status = 139 (128 + 11 = SIGSEGV)
```

What happened: the CPU tried to translate the address 0, found no valid mapping in the page table (page not present), and raised a page-fault exception. The kernel's handler checked that the address does not belong to any memory area of the process, so it was an illegal access; it sent the signal **SIGSEGV (11)** to the process, whose default action is to terminate and possibly produce a core dump. The shell then reported the exit status 139 = 128 + 11.

3.4 Segmentation with paging (Intel x86)



In the x86 architecture the logical address is a segment selector and an offset. The **segmentation unit** uses the selector to find the segment descriptor (base, limit, access rights) in the descriptor table, checks the limit and adds the base to get a **linear address**. The **paging unit** then splits the linear address into page-directory index, page-table index and offset and translates it through the page tables (using the TLB) into the physical address. Segmentation gives logical division and protection; paging gives fixed-size allocation without external fragmentation.

3.5 Segments of a running Linux program (/proc/<pid>/maps)

```
55863d994000-55863d995000 r--p /home/claude/a3/selfmaps     ELF header (read-only)
55863d995000-55863d996000 r-xp /home/claude/a3/selfmaps     code segment (read +
execute)
```

```
55863d996000-55863d997000 r--p /home/claude/a3/selfmaps     read-only data
55863d998000-55863d999000 rw-p /home/claude/a3/selfmaps     data segment (global
variables)
55867622c000-55867624d000 rw-p [heap]                       heap (grows up)
7ffd96cc0000-7ffd96ce1000 rw-p [stack]                      stack (grows down)
```

Linux does not use hardware segments for this (the x86-64 base is zero), but each memory area has its own permissions: code is r-x, data is rw-, the stack and heap are rw-; so the programmer's segments are still visible.

3.6 Evaluation

- **Programmer's view:** a program is a set of named parts – main program, functions, stack, heap, arrays. Segmentation maps each part to a segment of its own size, so the memory view matches the program structure, and an address (segment, offset) is meaningful.
- **Protection and sharing:** each segment has its own limit and permissions (read-only for code, read-write for data); a code segment can be shared by all students running the same compiler by putting the same base and limit into their segment tables; one table entry protects the whole segment.
- **External fragmentation:** segments have different sizes and must be placed in contiguous memory, so holes appear as segments come and go. In my simulation 20 random segments (10–99 KB) were loaded into 1,000 KB (first fit; 18 fitted and 992 KB were used); then 10 segments were removed. The result was **514 KB free in 10 holes, the largest hole only 96 KB; external fragmentation = 1 – 96/514 = 81.3%**, and a new 150 KB segment could not be loaded although 514 KB were free.

3.7 Comparison of pure paging, pure segmentation and segmented paging

Criterion	Pure paging	Pure segmentation	Segmented paging
Fragmentation	Internal only (up to a page per process or segment)	External (81% in my test)	Internal only (the last page of each segment)
Programmer's view	Hidden – one linear address space	Visible – segments	Visible
Allocation	Any free frame – simple	Contiguous hole – needs fit algorithm and compaction	Any free frame
Table size (e.g. a 1 MB program with code 300 KB, data 200 KB, stack 100 KB, heap 424 KB)	256 entries (1 MB / 4 KB)	4 entries (one per segment)	4 segment entries + about 256 page entries
Sharing / protection	Per page; code of several pages needs several entries	Per segment: easy	Per segment and per page

3.8 Conclusion – why modern operating systems rely mainly on paging

Paging has **no external fragmentation** and allocates memory in fixed-size units that fit any free frame, which makes allocation simple and fast, and it supports virtual memory (any page can be on the disk) easily. Segmentation needs contiguous holes of different sizes, suffers from external fragmentation and compaction, and makes virtual memory harder. Therefore Linux

and Windows use paging with a "flat" address space; the segment registers in x86-64 are kept at base 0, and the protection that segments gave (read-only code, no-execute data, stack) is provided by permission bits in the page-table entries.

3.9 Alternative concept – pure paging instead of segmentation

Criterion	Segmentation	Pure paging
Fragmentation	External (81.3% in my test) plus none inside	Internal only: about half a page per program (≈2 KB for 4 KB pages)
Table size	Few entries (one per segment, for example 4) – O(number of segments)	One entry per page: 256 entries for 1 MB (4 B each = 1 KB), and a multi-level table to keep it small for large address spaces – O(pages)
Translation time	One comparison and one addition (O(1)) with a table in registers	TLB hit: almost free; miss: 1–4 table accesses
Code sharing and protection	One segment-table entry for the whole code; limit and rights for the whole segment	Each page of the code needs an entry (read-only, shared frames); more entries, but still easy and the OS can protect each page separately

Pure paging trades a larger table and a TLB for the removal of external fragmentation, which is the better trade for general-purpose systems.

Q4 – Virtual Memory and Demand Paging

Q4 (K5 – Evaluate) **[8 Marks]**

Context: A radiologist runs an image-processing program that needs 24 GB on a laptop with 16 GB RAM.

Implementation: steps of a page fault; effective access time for fault rates 1/1,000, 1/100,000 and 1/1,000,000 (memory 100 ns, fault service 8 ms); copy-on-write and fork(); thrashing and the working-set model.

Evaluation: how demand paging lets 24 GB run in 16 GB; maximum fault rate for a 10% slow-down; equal and proportional frame allocation; how the OS detects and stops thrashing.

Experiments: program touching memory sequentially and randomly with page-fault counts; run time as memory grows beyond free RAM; copy-on-write measurement; plot of page faults against frames.

Conclusion: more RAM or better locality?

Alternative Concept: a machine with 32 GB RAM instead of demand paging.

Answer:

4.1 Steps in handling a page fault

- The process references a page; the MMU finds the valid bit = invalid (page not in memory) in the page table and **traps** to the OS.
- The OS checks the process's page table/internal table: if the reference is illegal, the process is terminated; if the page is legal but not in memory, continue.
- The OS finds a **free frame** (if none is free, a victim page is chosen by a page-replacement algorithm and written to the disk if it is dirty).
- The OS schedules a **disk read** of the page into the frame; the process waits (blocked) and another process can use the CPU.

- When the disk I/O finishes, the OS updates the page table (frame number, valid bit = 1) and the internal tables.
- The process is made ready again; the **instruction that caused the fault is restarted**, now with the page in memory.

4.2 Effective access time for the given fault rates

EAT = (1 – p) × memory access time + p × page-fault service time = (1 – p) × 100 ns + p × 8,000,000 ns.

Page-fault rate p	EAT	Slow-down compared with 100 ns
1 in 1,000 (10^{-3})	0.999 × 100 + 0.001 × 8,000,000 = 8,099.9 ns	81 times slower
1 in 100,000 (10^{-5})	0.99999 × 100 + 0.00001 × 8,000,000 = 180.0 ns	1.8 times slower
1 in 1,000,000 (10^{-6})	0.999999 × 100 + 0.000001 × 8,000,000 = 108.0 ns	1.08 times slower (8%)

A very small fault rate is needed: one fault in a thousand accesses makes the machine 81 times slower, because a page fault costs 80,000 times more than a memory access.

4.3 Maximum acceptable page-fault rate for a 10% slow-down

A 10% slow-down means EAT ≤ 110 ns: 100 + p × (8,000,000 – 100) ≤ 110 → p ≤ 10 / 7,999,900 = **1.25×10^{-6}**, i.e. no more than **about one page fault in 800,000 memory accesses**.

4.4 How a 24 GB program can run in 16 GB (demand paging)

In demand paging, a page is brought into memory only when it is referenced; the rest of the process stays in the backing store (swap space or the file). The logical address space (24 GB) can be larger than the physical memory (16 GB) because at any moment the program uses only a part of it – its **working set** – because of locality of reference. The image program, for example, processes one region (slice or tile) at a time; the pages of the rest are on the disk. When a free frame is needed, the OS replaces a page that is not in use (usually the least recently used). The program therefore runs correctly, but its speed depends on the page-fault rate (4.2): if the working set fits into 16 GB, the faults are rare and the program is almost as fast as in a larger memory.

4.5 Copy-on-write and fork()

When fork() creates a child, the OS does not copy the pages of the parent; the parent and the child share the same frames, marked **read-only (copy-on-write)**. When either process writes to a page, a protection fault occurs; the OS makes a **private copy** of that page for the writer and then the write continues. Only the pages that are modified are copied, so fork() is fast and most pages (especially the code) are never copied – and when the child calls exec(), nothing needs to be copied at all.

Measurement 1 (speed of fork()): a process with 1 GB of touched memory called fork(); the fork() took **6.84 ms** (only page tables were copied, no data).

Measurement 2 (memory before and after writing): a parent wrote 200 MB, forked, and the child read /proc/self/smaps_rollup before and after writing 100 MB:

State of the child	Rss	Pss	Private_Dirty	Shared_Dirty
Right after fork	205,604 kB	102,641 kB	40 kB	204,876 kB
After writing 100 MB	205,920 kB	153,975 kB	102,448 kB	102,468 kB

Right after fork almost all pages are **shared** (Shared_Dirty 204,876 kB, Private_Dirty only 40 kB) and Pss is about half of Rss, because the memory is divided between the two processes. After the child wrote 100 MB, about 100 MB (102,448 kB) became **private** to the child: copies were made only for the pages that were written, while the other 100 MB remained shared. This proves copy-on-write.

4.6 Thrashing and the working-set model

Thrashing happens when a process (or the system) does not have enough frames for its active pages: it faults continuously, replaces pages that it needs again immediately, and spends more time paging than executing. The CPU utilisation falls; if the OS then sees low CPU use and starts more processes, the situation becomes worse.

Working-set model: based on locality. The working set $WS(t, \Delta)$ of a process is the set of pages referenced in the last Δ memory references. If the total size of the working sets of all processes ($D = \Sigma\ WSS_i$) is larger than the number of frames m, thrashing will occur; the OS then suspends one or more processes (swaps them out) until $D \leq m$. The OS gives each process enough frames for its working set; if the working set cannot be satisfied the process is not run.

4.7 Allocation of frames, and how thrashing is detected and stopped

- **Equal allocation:** m frames are divided equally among n processes (m / n each). Example: 62 frames for 2 processes = 31 each; simple, but a small process wastes frames.
- **Proportional allocation:** frames are given in proportion to the size s_i of the process: $a_i = s_i / S \times m$. Example: 62 free frames, process 1 of 10 pages and process 2 of 127 pages ($S = 137$): $a_1 = 10/137 \times 62 \approx 4$ frames and $a_2 = 127/137 \times 62 \approx 57$ frames. (Priority can also be included.)
- **Detection:** the OS monitors the CPU utilisation and the page-fault rate. A high paging rate with a low CPU utilisation (and growing disk queue) signals thrashing. With the **page-fault-frequency (PFF)** method the OS sets an upper and a lower limit for the fault rate of each process: if the rate exceeds the upper limit the process gets another frame (or, if none is free, it is suspended); if it falls below the lower limit a frame is taken away.
- **Stopping:** reduce the degree of multiprogramming (suspend/swap out a process), use local replacement so that one process cannot steal the frames of another, and give each process its working set.

4.8 Experiment – sequential and random touching of memory

Program q4t.c allocates memory with mmap and writes one byte in each 4 KB page, in sequential order and in a scrambled (random-like) order, counting the page faults (getrusage) and the time. The test machine has about 4 GB of RAM and no swap, so the sizes were 64 MB to 2 GB:

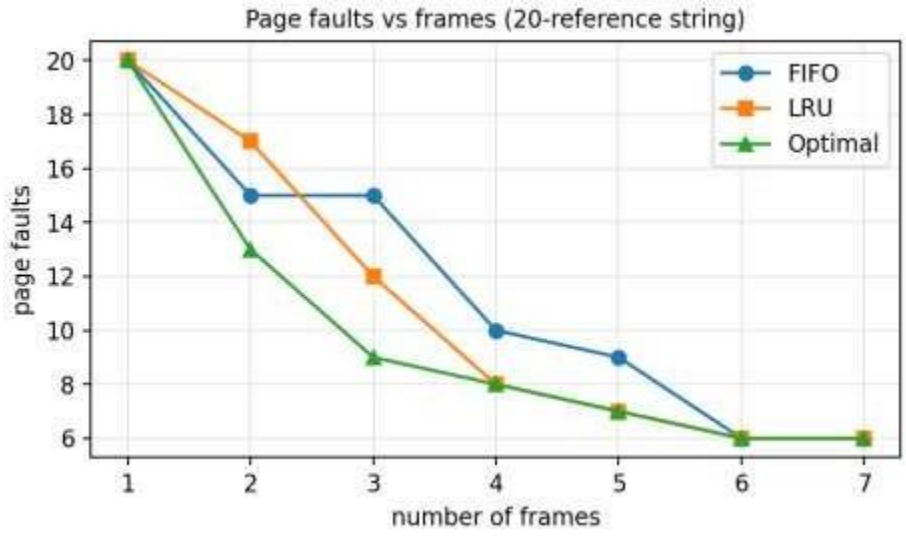
Size	Order	Minor page faults	Pages (size / 4 KB)	Time (s)
64 MB	sequential	16,386	16,384	0.039
64 MB	random	16,384	16,384	0.027
256 MB	sequential	65,536	65,536	0.141
256 MB	random	65,536	65,536	0.099
1024 MB	sequential	262,144	262,144	0.588
1024 MB	random	262,144	262,144	0.446
2048 MB	sequential	524,288	524,288	1.039
2048 MB	random	524,288	524,288	0.802

Results: the number of page faults is equal to the number of pages touched (one fault for each page on its first use – demand paging) and it is the same for the sequential and the random order, because each page is touched once. The time is proportional to the number of faults (about 2 µs per fault in the sequential test: 1.039 s / 524,288 ≈ 2 µs, including the zeroing of the page). With more than 1 GB the time continues to grow linearly; the faults were "minor" (the page is created in memory without disk I/O). The difference between sequential and random order here is small, because there was enough free RAM.

Limitation: the test machine has no swap and I could not set a memory limit or run a virtual machine, so I could not measure the run time for a program larger than RAM (major faults). For this case I used the calculation in 4.2: when the program does not fit, every fault costs about 8 ms, which is about 4,000 times more than the 2 µs minor fault measured here. With a reasonably small fault rate, such as 10^{-6}, the program is 8% slower; with 10^{-3} it is 81 times slower. On a computer with a swap file or a memory-limited container, the experiment should be repeated by increasing the size past the RAM and the time will show a sudden increase (a "knee") when the memory is exceeded.

4.9 Plot of page faults against the number of frames

For the reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 (20 references):



Frames	1	2	3	4	5	6 and 7
FIFO	20	15	15	10	9	6
LRU	20	17	12	8	7	6
Optimal	20	13	9	8	7	6

The faults fall as frames are added, until the 6 frames needed for the 6 different pages (7, 0, 1, 2, 3, 4) are available: then only the first-use faults remain. The curve shows the "knee": the program runs well with enough frames for its working set; too few frames cause many faults (thrashing).

4.10 Conclusion – more RAM or better locality?

Both help, but **improving locality is the better first step**, and extra RAM is the better second step if the working set is still larger than the memory. Reason: the fault rate must be below about 1 in 800,000 accesses for a 10% slow-down; a program with poor locality (jumping across 24 GB) needs a fault rate far above this, so it would be thrashing even in 16 GB, and adding 8 GB would not help unless the whole 24 GB fits. If the program is changed to process the image in tiles/blocks (so that its working set is a few hundred MB) it runs at almost full speed in 16 GB or even less, at no cost. If the program cannot be changed, or its working set really is larger than 16 GB, buying more RAM is the only fix. So the radiologist should first restructure the access pattern (blocking, sequential access) and then add RAM if needed.

4.11 Alternative concept – 32 GB of RAM instead of demand paging

Criterion	Demand paging in 16 GB	32 GB RAM (all 24 GB resident)
Effective access time	$p = 10^{-5} \rightarrow 180$ ns; $p = 10^{-6} \rightarrow 108$ ns (8 ms per fault)	About 100 ns – no major faults after the first load (the program's first touch still creates the pages)
Slow-down	1.8 times ($p = 10^{-5}$), 1.08 times ($p = 10^{-6}$), 81 times ($p = 10^{-3}$)	1.0 – no paging
Cost	No extra hardware; the cost is the extra run time and the disk wear	A one-time cost for 16 GB more RAM (if the laptop supports it); the run time is the best possible

(The measured minor-fault rate cannot be used here, because the minor faults of the first touch occur in both cases; the major-fault rate is not measurable on a machine without swap, so the rates 10^{-5} and 10^{-6} are examples.) If the radiologist runs this program every day and each run is slowed even by 80% ($p = 10^{-5}$), the lost time over a year is larger than the price of the memory; for an occasional run, demand paging is enough. The extra memory is also useful for all other programs.

Q5 – Page-Replacement Techniques

Q5 (K5 – Evaluate) **[8 Marks]**

Context: A web server keeps frequently used pages in memory; when memory is full the OS must choose which page to remove.

Implementation: FIFO, LRU and Optimal; faults for a reference string with 3 and 4 frames; Belady's anomaly; clock algorithm.

Evaluation: why Optimal is best but not implementable; why LRU has no Belady's anomaly; hardware support for LRU and how clock avoids it; comparison with strong locality.

Experiments: reference strings with locality and random; faults for 3 to 10 frames; plot; running time for 100,000 references; hand calculation.

Engineering decision: recommended policy.

Complete the Code: TODO 1 (LRU); explain Belady's anomaly.

Alternative Concept: clock (second-chance) instead of exact LRU.

Answer:

5.1 Completed program (TODO 1 – LRU)

```
#include <stdio.h>

#define MAXF 8

int find(const int f[], int n, int page)
{
    for (int i = 0; i < n; i++) if (f[i] == page) return i;
    return -1;
}

int fifo(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, next = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        if (find(f, used, ref[i]) >= 0) continue;
        faults++;
        if (used < frames) f[used++] = ref[i];
        else { f[next] = ref[i]; next = (next + 1) % frames; }
    }
    return faults;
}

int lru(const int ref[], int len, int frames)
{
    int f[MAXF], last[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        int k = find(f, used, ref[i]);
        if (k >= 0) { last[k] = i; continue; }
        faults++;
        if (used < frames) { f[used] = ref[i]; last[used] = i; used++; }
        else {
            int victim = 0;
            for (int j = 1; j < frames; j++) if (last[j] < last[victim]) victim =
j;
            f[victim] = ref[i];
            last[victim] = i;
        }
    }
    return faults;
}

int optimal(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        if (find(f, used, ref[i]) >= 0) continue;
        faults++;
        if (used < frames) { f[used++] = ref[i]; continue; }
        int victim = 0, farthest = -1;
        for (int j = 0; j < frames; j++) {
            int next = len;                    /* never used again */
            for (int t = i + 1; t < len; t++) if (ref[t] == f[j]) { next = t;
break; }
            if (next > farthest) { farthest = next; victim = j; }
```

```
            }
            f[victim] = ref[i];
        }
        return faults;
}

int main(void)
{
        int ref[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
        int len = 20;
        for (int fr = 3; fr <= 4; fr++)
            printf("%d frames: FIFO %d, LRU %d, Optimal %d\n", fr, fifo(ref, len, fr),
                   lru(ref, len, fr), optimal(ref, len, fr));
        int b[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
        printf("Belady string, FIFO: 3 frames %d faults, 4 frames %d faults\n",
               fifo(b, 12, 3), fifo(b, 12, 4));
        return 0;
}
```

The TODO: if a frame is free, put the page in it; otherwise find the frame with the smallest last[] (the least recently used page) and replace it; in both cases set last[frame] = i. Output (same as expected):

```
3 frames: FIFO 15, LRU 12, Optimal 9
4 frames: FIFO 10, LRU 8, Optimal 8
Belady string, FIFO: 3 frames 9 faults, 4 frames 10 faults
```

5.2 Hand calculation for one string (verification)

String 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 with 3 frames. **LRU** (frame contents from the least recently used to the most recently used; F = fault):

Ref	**7**	**0**	**1**	**2**	**0**	**3**	**0**
Frames	7	7 0	7 0 1	0 1 2	1 2 0	2 0 3	2 3 0
	F	F	F	F	hit	F	hit
Ref	4	2	3	0	3	2	1
Frames	3 0 4	0 4 2	4 2 3	2 3 0	2 0 3	0 3 2	3 2 1
	F	F	F	F	hit	hit	F
Ref	2	0	1	7	0	1	
Frames	3 1 2	1 2 0	2 0 1	0 1 7	1 7 0	7 0 1	
	hit	F	hit	F	hit	hit	

Faults: 7, 0, 1, 2, 3, 4, 2, 3, 0, 1, 0, 7 = **12 faults**, as in the program. For **FIFO** with 3 frames (oldest page replaced first) the faults are at references 1–4, 6–11, 14, 15, 18, 19 and 20 → **15 faults** (the program gives 15). **Optimal** replaces the page whose next use is farthest in the future and gives 9 faults. The hand calculation and the program agree.

5.3 Belady’s anomaly with FIFO

String 1 2 3 4 1 2 5 1 2 3 4 5; contents of the frames after each reference (* = fault):

3 frames	**1* | 12* | 123* | 423* | 413* | 412* | 512* | 512 | 512 | 532* | 534* | 534 → 9 faults**
4 frames	1* | 12* | 123* | 1234* | 1234 | 1234 | 5234* | 5134* | 5124* | 5123* | 4123* | 4523* → 10 faults

Why FIFO has more faults with 4 frames than with 3: FIFO removes the oldest page and does not look at how recently it was used. With 4 frames the set of pages in memory is different, not a superset of the set with 3 frames: for example with 4 frames the pages 1 and 2 (which are used again soon) are the oldest and are removed just before they are needed (the faults for 1, 2, 3, 4, 5 happen one after another), whereas with 3 frames the hits at 1 and 2 occur by chance. This effect (more frames giving more faults) is **Belady's anomaly**.

Why LRU cannot behave like this: LRU is a **stack algorithm**. With n frames, the pages in memory are always the n most recently used pages; with n + 1 frames they are the n + 1 most recently used pages, which always include the n pages of the smaller memory (the set for n frames is a subset of the set for n + 1 frames). So any reference that is a hit with n frames is also a hit with n + 1 frames, and the number of faults can never increase when frames are added. The same is true for Optimal. In my results LRU gave 12 faults with 3 frames and 8 with 4, never more.

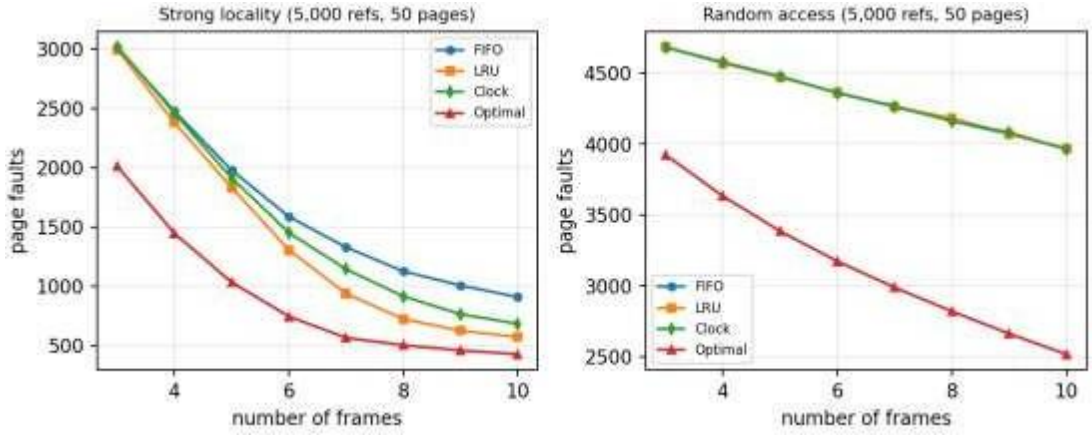
5.4 Optimal, LRU hardware and the clock algorithm

- **Optimal (MIN):** replace the page that will not be used for the longest time. It gives the **lowest possible number of faults** for any string and is used as a standard of comparison. It **cannot be implemented** in a real system, because it needs to know the future page references.
- **LRU needs hardware support:** the system must know the order of the last use of the pages, so on **every memory reference** it must update either a time-stamp (counter) in the page-table entry, or move the page to the head of a stack/list. This is too slow to do in software for each memory access and needs special hardware, and finding the oldest page needs a search of the table (O(frames) in my program).
- **Clock (second chance):** each frame has one **reference bit** that the hardware sets to 1 when the page is used (one cheap operation). The frames are arranged in a circle with a pointer (the clock hand). To find a victim: if the bit at the hand is 0, replace this page; if it is 1, set it to 0 (second chance) and advance the hand. A page that is used again before the hand returns survives; a page not used is replaced. It approximates LRU without time-stamps or lists.

5.5 Experiment – locality against random access (3 to 10 frames)

Program q5x.c generates two reference strings with 5,000 references on 50 pages: (a) **strong locality** – a working set of 6 pages that moves every 250 references, with 90% of the references inside the working set; (b) **random** – uniform over the 50 pages. Page faults:

Frames	FIFO (loc.)	LRU (loc.)	Clock (loc.)	Opt (loc.)	FIFO (rand.)	LRU (rand.)	Clock (rand.)	Opt (rand.)
3	3015	2996	3021	2012	4683	4687	4681	3923
4	2477	2382	2460	1447	4577	4570	4577	3632
5	1980	1834	1913	1034	4477	4472	4477	3385
6	1587	1307	1453	746	4361	4363	4361	3173
7	1330	939	1145	566	4269	4264	4264	2988
8	1127	724	915	503	4164	4183	4163	2820
9	1007	626	765	460	4070	4069	4082	2662
10	910	573	683	427	3969	3969	3964	2517



Observations: (1) with strong locality the faults fall quickly as frames are added and LRU is clearly better than FIFO (at 7 frames 939 against 1,330, i.e. 29% fewer) and Clock lies between them (1,145); (2) with random access the three practical algorithms are practically the same (about 4,000–4,700 faults), because no algorithm can predict a random future – only Optimal is better (it knows the future); (3) the faults always decrease when frames are added for LRU and Optimal.

5.6 Running time for 100,000 references

Algorithm	Time (6 frames, locality string)	Page faults	Remark
FIFO	0.00108 s	30,206	Cheapest
LRU	0.00146 s	24,106	Fewest faults of the practical algorithms
Clock	0.00164 s	27,475	In software; in a real system it is cheaper (one bit)
Optimal	0.00829 s	13,741	Looks into the future: about 8 times more time, not possible in practice

All the practical algorithms need only about 1–2 ms for 100,000 references in a simulation, so the CPU time is negligible compared with the cost of one page fault (milliseconds for a disk, even though a hit costs nanoseconds); what matters is the number of faults, and in a real OS the hardware cost of updating the LRU information on every access.

5.7 Alternative concept – clock instead of exact LRU

```
/* on every reference to a page in a frame */    frame.ref = 1;
/* hardware does this */
/* on a page fault */
while (frame[hand].ref == 1) { frame[hand].ref = 0; hand = (hand + 1) % N; }    /*
second chance */
replace frame[hand]; frame[hand].ref = 1; hand = (hand + 1) % N;
```

Criterion	Exact LRU	Clock
Time per reference	Update a time-stamp or move a list element on every access: O(1) per access but expensive (hardware or software overhead)	Set one bit: O(1), very cheap

Criterion	Exact LRU	Clock
Time to select a victim	O(frames) to find the oldest (or O(1) with a linked list)	Amortised O(1); at worst one full turn of the clock, O(frames)
Extra space per frame	A time-stamp (for example 32–64 bits) or two list pointers	1 bit (plus one hand pointer for the whole system)
Page faults (my simulation, locality string, 3 to 10 frames)	e.g. 1,307 at 6 frames; 573 at 10 frames	e.g. 1,453 at 6 frames (11% more); 683 at 10 frames (19% more) – always between FIFO and LRU

Clock gives fewer faults than FIFO and is within about 10–20% of LRU for the locality workload; with random access it is the same as LRU.

5.8 Engineering decision for the web server

Recommendation: the clock (second-chance) algorithm – the practical form of LRU – for the web server. The measurements show that for a workload with strong locality (which web content normally has: a few popular pages are requested very often) LRU has up to 37% fewer page faults than FIFO and Clock up to 25% fewer, the gain growing with the number of frames (for example at 7 frames: FIFO 1,330, Clock 1,145, LRU 939), and for a random workload no algorithm helps. Exact LRU is slightly better in the number of faults but needs the hardware or software to update an ordering on every access, which is too costly on a busy server; Clock needs only a reference bit and costs almost nothing. FIFO is the cheapest but it suffers from Belady's anomaly and keeps old pages that are still popular. Optimal cannot be used. If the server can add memory, the faults decrease rapidly (from 1,587 to 910 for FIFO between 6 and 10 frames), so adding enough frames for the popular pages is the first measure.

Q6 – Disk Management

Q6 (K5 – Evaluate) **[4 Marks]**

Context: The hospital records server must start reliably, survive a disk failure and have enough swap space for peak load.

Implementation: low-level formatting, partitioning, logical formatting; boot block and boot process; bad blocks; size and location of swap.

Evaluation: RAID 0, 1, 5, 10 (reliability, capacity, speed) and the choice for the server; swap on a separate disk or a swap file; partitions on Linux (lsblk, fdisk -l).

Experiments: partitions and swap of the Linux system; usable capacity of 6 × 4 TB for each RAID level; probability of data loss for RAID 1 and RAID 5; read speed on SSD and hard disk.

Conclusion: recommended storage configuration.

Alternative Concept: RAID 6 instead of RAID 5 or RAID 10.

Answer:

6.1 Low-level formatting, partitioning and logical formatting

- **Low-level (physical) formatting:** done by the manufacturer. The disk is divided into tracks and sectors, and each sector gets a header, a data area (usually 512 bytes or 4 KB) and a trailer with an error-correcting code (ECC).
- **Partitioning:** the OS divides the disk into one or more groups of cylinders/sectors called partitions (stored in a partition table – MBR or GPT); each partition can be treated as a separate disk (for example /boot, / and swap).

- **Logical formatting:** the OS creates a file system in a partition (for example ext4): the initial data structures – the free-space map, the empty directory, the inode table. (Raw I/O without a file system is used by some databases and by swap.)

6.2 Boot block and boot process

When the computer is switched on, a small bootstrap program in firmware (BIOS or UEFI) runs, tests the hardware, and loads the **boot block** (the first sector of the disk, the MBR; with UEFI the EFI system partition) into memory and executes it. The boot block is a small program that finds and loads the larger boot loader (for example GRUB) from the boot partition; the boot loader loads the **kernel** (and the initial RAM disk) from the file system into memory and starts it; the kernel initialises the devices, mounts the root file system and starts the first process (init/systemd), which starts the services. For the records server a mirrored boot partition is needed so that the machine can start after a disk failure.

6.3 Bad blocks

Sectors may be damaged. With **sector sparing (forwarding)**, the disk controller keeps spare sectors; when a bad sector is found, the controller replaces it logically by a spare sector, and the OS does not know. With **sector slipping**, the sectors after the bad one are moved down one position to use the spare. Soft errors are corrected by the ECC; hard errors are marked as bad (in older systems the OS, for example the formatting or fsck/badblocks tools, marks the block in the file system so that it is never allocated). On a records server the SMART monitoring of the disks (reallocated sector count) is used to replace a disk before it fails.

6.4 Swap space – size and location

Assumption (the question gives no figures): the records server has 64 GB of RAM. A common guide is swap = 0.5 × RAM for a large-memory server (about 16 GB for 64 GB); the swap should be sufficient for the peak load and for rarely used pages, but the server should not normally swap (heavy swapping means thrashing – add RAM instead).

Criterion	Swap partition / separate disk	Swap file in a file system
Speed	Slightly faster – direct, contiguous, no file-system overhead; on a separate disk it does not compete with the database I/O	Slightly slower; the file may be fragmented
Flexibility	Fixed size – changing needs re-partitioning	Easy to grow, shrink or add (mkswap, swapon)
Reliability	If the swap disk fails the system can crash – so it should be mirrored (RAID 1)	Is as reliable as the file-system volume (RAID)

Decision: a 16 GB swap area on a **mirrored (RAID 1) fast SSD pair separate from the data disks**, so that paging does not slow the database and a disk failure does not crash the system; extra swap files can be added later for an unexpected peak.

6.5 Partitions and swap of the test Linux system

Commands and output on the test machine (a virtual machine):

```
$ lsblk -o NAME,SIZE,TYPE,MOUNTPOINTS
NAME   SIZE TYPE MOUNTPOINTS
vda    256G disk /                    <- one virtual disk, used as a whole
vdb   12.5M disk /opt/rclone
```

```
vdc    696K disk /mnt/skills/public
$ cat /proc/swaps
Filename    Type    Size    Used    Priority          <- empty: no swap is configured
$ free -m
Mem:  3995 ...    Swap: 0 0 0
$ df -h /
/dev/vda  252G  8.6G  10G  47% /
$ fdisk -l
fdisk: not found (not installed in this container)
```

On this virtual machine the disk vda has no partition table; the whole disk is the root file system, and **there is no swap** (Swap: 0). On a normal Linux server lsblk shows the tree disk → partition (sda1, sda2…) → RAID (md0) or LVM volumes and their mount points, fdisk -l shows the partition table (start, end, size, type) and swapon --show / cat /proc/swaps shows the swap areas. A disk is seen by Linux as a device file (/dev/sda, /dev/nvme0n1) and its partitions as /dev/sda1, /dev/sda2.

6.6 RAID levels

Level	Principle	Capacity (n disks)	Failures survived	Speed
RAID 0	Striping: blocks spread over the disks, no redundancy	n × disk	0 (any failure loses all)	Fastest read and write
RAID 1	Mirroring: every block written to two disks	1 disk of n (for a pair)	1 per mirror	Fast read; write as one disk
RAID 5	Striping with one distributed parity block	(n – 1) × disk	1	Fast read; write is slow (4 I/Os, parity)
RAID 10	Striped set of mirrored pairs (1+0)	n / 2 × disk	1 per pair (at least 1, up to n/2)	Fast read and write; fast rebuild

6.7 Usable capacity of 6 disks of 4 TB

RAID level	Usable capacity	Remark
RAID 0	6 × 4 = 24 TB	No protection
RAID 1 (3 pairs; the pairs are combined as RAID 10)	3 × 4 = 12 TB (one pair = 4 TB)	50% of the raw capacity
RAID 5	(6 – 1) × 4 = 20 TB	1 disk of capacity used for parity
RAID 6 (alternative concept)	(6 – 2) × 4 = 16 TB	2 disks of capacity used for parity
RAID 10	(6 / 2) × 4 = 12 TB	50% of the raw capacity

6.8 Probability of data loss for RAID 1 and RAID 5

Assumptions (not given in the question): annual failure rate (AFR) of a disk = 3%; a failed disk is replaced and rebuilt in 24 hours; failures are independent. The probability that one particular disk fails in the 24-hour rebuild window is p = 0.03 × 24 / 8,760 = **8.22×10^{-5}**. Data are lost if another disk needed for the rebuild fails during it:

Configuration	P(data loss | one disk has failed)	Meaning
RAID 1 (a mirrored pair)	$p = 8.22 \times 10^{-5}$	The one remaining disk must not fail: 1 in 12,167
RAID 5 (6 disks)	$1 - (1 - p)^5 = 4.11 \times 10^{-4}$	Any of the 5 remaining disks failing: 1 in 2,434
RAID 6 (6 disks)	about $6p^2 = 4.05 \times 10^{-8}$	Two more disks of the 4 remaining must fail: 1 in 24.7 million

RAID 5 with 6 disks is **5 times more likely** to lose data during a rebuild than a RAID 1 pair, because every one of the other disks must survive the whole rebuild (a large 4 TB disk takes many hours). Per year, the probability of a data loss for the array is approximately (number of disks × AFR) × P(loss | one failure): RAID 1 pair 4.9×10^{-6}, RAID 5 (6 disks) 7.4×10^{-5}. (The unrecoverable read errors during the rebuild, which make RAID 5 with large disks even riskier, are not included.)

6.9 Read speed of a file

Only one (virtual) disk is available on the test machine, so an SSD could not be compared with a hard disk. A 512 MB file was written and read with dd:

```
write, direct I/O  : 455 MB/s
read, direct I/O   : 2.3 GB/s
read, with the page cache: 1.8 GB/s
```

These values belong to a virtual disk with the host's caching and are not typical for a physical hard disk. For comparison, typical values are: hard disk 100–200 MB/s sequential and about 8–10 ms per random access; SATA SSD about 500 MB/s and 0.1 ms; NVMe SSD 2–7 GB/s. The random access, which a records database does, is faster on an SSD by about 100 times.

6.10 Conclusion – recommended storage configuration

I recommend **RAID 10 over the six 4 TB disks (12 TB usable)** for the patient-record data, a **mirrored pair of SSDs (RAID 1)** for the operating system, boot partition and the 16 GB swap, and regular off-site backups. Reasons: the records server is mostly a database with many small random writes, for which RAID 10 is the fastest redundant level (2 I/Os per write against 4 for RAID 5), it keeps working after a disk failure and rebuilds quickly by copying one disk (a short rebuild window means a smaller probability of a second failure), and it can survive up to three failures if they are in different pairs. RAID 5 gives more capacity (20 TB) but a slower write and 5 times higher probability of a loss during a rebuild; RAID 0 is not allowed for medical records. The price is only 50% usable capacity. RAID protects against disk failure, not against deletion, corruption or a fire, so backups are still needed.

6.11 Alternative concept – RAID 6

Criterion	RAID 6 (6 disks)	RAID 5 / RAID 10
Usable capacity of 6 × 4 TB	16 TB	RAID 5: 20 TB; RAID 10: 12 TB
Time to write a block	Small write = 6 I/Os (read old data, P, Q; write new data, P, Q) and two parity calculations (XOR and a Reed–Solomon code): O(1) per block but a larger constant	RAID 5: 4 I/Os and one XOR; RAID 10: 2 writes and no parity calculation

Criterion	RAID 6 (6 disks)	RAID 5 / RAID 10
Failures survived	Any 2 disks	RAID 5: 1; RAID 10: 1 to 3 (depending on which disks)
P(loss | one failure) for the rebuild	4.05×10^{-8}	RAID 5: 4.11×10^{-4}

RAID 6 is the right choice if capacity matters more than write speed (an archive of old scans, for example); for the transaction-oriented records server RAID 10 remains better because of the write performance, while RAID 6 would be the second choice.

Q7 – Design a Memory-Management Plan from First Principles

> **Q7 (K5 – Evaluate)** **[10 Marks]**
>
> **Context:** A smart-factory edge device with 2 GB RAM and no swap disk; it runs a real-time control task that must never be delayed by a page fault, a machine-learning task that needs 1.5 GB, and a logging task. More tasks will be added later.
>
> **Stage 1 (2):** interpret the problem – memory needs, timing constraints, page-fault tolerance, measurable requirements.
>
> **Stage 2 (3):** compare contiguous allocation, paging, segmentation and demand paging for each task; how memory is locked (mlock()); comparison table.
>
> **Stage 3 (3):** program that locks the memory of a time-critical loop and measures the worst-case delay with and without locking; simulate the three tasks with a page-replacement simulator; verify that the real-time task is never delayed.
>
> **Stage 4 (2):** justify with measurements; what changes with a fourth task of 500 MB; one limitation.
>
> **Alternative Concept:** a real-time OS with no virtual memory instead of Linux with locked memory.

Answer:

Stage 1 – Interpret the Problem (2 Marks)

The question does not give some figures, so I made these assumptions: the control task needs 64 MB (code, stack and buffers), the logging task 128 MB, and the Linux kernel and system services 256 MB. Sizes and timing:

Task	Memory	Timing constraint	Page-fault tolerance
Real-time control	64 MB (assumed)	Hard deadline: each control cycle must end within a fixed time (assumed 1 ms period); worst-case delay is what matters, not the average	None – a major fault costs about 8 ms (more than the 1 ms deadline); even a minor fault adds tens of µs
Machine learning	1.5 GB (1,536 MB)	Throughput-oriented; an inference may take seconds; no hard deadline	Tolerates some faults, but a high rate makes it much slower; it is the largest user of memory
Logging	128 MB (assumed)	Soft; delays of seconds are acceptable (buffered writes)	Tolerates page faults and can be delayed
OS and services	256 MB (assumed)	—	—

Total demand: 64 + 1,536 + 128 + 256 = **1,984 MB out of 2,048 MB**, leaving only 64 MB free. There is no swap disk, so the anonymous (heap/stack) pages cannot be written out: if memory is exhausted the kernel can only drop file-backed pages (code, a memory-mapped model file) or kill a process (the out-of-memory killer).

Measurable requirements:

- R1: the control task has **0 page faults** in its loop after start-up (all its memory is locked in RAM before the loop starts).
- R2: the worst-case time of one control cycle is below the deadline (assumed 1 ms) in every measured run; measured as the maximum of at least 4,096 cycles.
- R3: locked memory (control task) ≤ 64 MB and total memory demand of all tasks ≤ 2,048 – 256 (OS) = 1,792 MB.
- R4: the machine-learning task uses at most 1,536 MB and the logging task at most 128 MB; a new task may be added only if the total stays within 1,792 MB (or the other tasks are changed).

Stage 2 – Compare Candidate Mechanisms (3 Marks)

Mechanism	Control task	ML task (1.5 GB)	Logging task	Overall
Contiguous allocation	Predictable, no faults – but needs one block of fixed size; poor flexibility	Needs a 1.5 GB contiguous hole; external fragmentation makes it hard on a long-running device	Wasteful; adding tasks later is difficult	Only suitable if the memory map is fixed forever
Paging	Good if the pages are locked – no external fragmentation; page-table walks add small variable delay (TLB misses)	Good – any free frames can be used; 4 KB (or huge 2 MB) pages	Good	Basis of the design (Linux)
Segmentation	Protects code/data separately but external fragmentation; not used by Linux	Contiguous segment of 1.5 GB – the same problem as contiguous allocation	Not needed	Rejected (see Q3)
Demand paging	Not acceptable – a first touch or eviction causes a fault of unpredictable delay	Acceptable – the model file can be memory-mapped and loaded as needed; it allows a model bigger than the free memory, but slows it down when memory is short	Good – saves memory because only used pages are loaded	Use only for ML and logging

How the memory of the real-time task is locked: (1) call **mlockall(MCL_CURRENT | MCL_FUTURE)** at the start of the control process (or mlock() on its buffers), which locks all the present and future pages in RAM so that they can never be paged out; (2) **pre-fault** the memory before the loop: touch every page of the buffers and of the stack (memset), so that the pages already exist when the loop starts – mlock does this for the pages it locks; (3) do not allocate memory (malloc/free) or load libraries inside the loop; (4) raise the limit RLIMIT_MEMLOCK (ulimit -l shows 8192 kB on the test machine, so a normal user could not lock 64 MB; my program could, because it ran as root, which has the CAP_IPC_LOCK permission); (5) run the task with a real-time priority (SCHED_FIFO) and pin it to a core; (6)

put the other tasks in a cgroup with a memory limit so that they cannot take the memory of the control task.

Stage 3 – Implement and Verify (3 Marks)

3.1 Program: locked and unlocked time-critical loop

The control loop makes 4,096 cycles; in each cycle it touches four new pages of a 64 MB buffer (so that, without locking, the cycle meets page faults) and measures the time of the cycle in microseconds. With the argument 1 the buffer is locked with mlock() and pre-faulted before the loop starts.

```
#define _POSIX_C_SOURCE 200809L
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <sys/mman.h>
#include <sys/resource.h>
static double now_us(void){struct timespec
t;clock_gettime(CLOCK_MONOTONIC,&t);return t.tv_sec*1e6+t.tv_nsec/1e3;}
#define BUF (64UL*1024*1024)
#define CYCLES 4096          /* one cycle touches 4 new pages of the 64 MB working
buffer */
int main(int argc,char**argv){
  int lock=argc>1&&argv[1][0]=='1';
  char*buf=mmap(NULL,BUF,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0);
  if(lock){ if(mlock(buf,BUF)!=0){perror("mlock");return 1;} memset(buf,0,BUF); }
/* lock + pre-fault before the loop starts */
  struct rusage a,b;getrusage(RUSAGE_SELF,&a);
  double worst=0,sum=0,tmin=1e9;
  for(int c=0;c<CYCLES;c++){
    double t=now_us();
    for(int k=0;k<4;k++)buf[((size_t)c*4+k)*4096]++;    /* the time-critical work
*/
    double d=now_us()-t;if(d>worst)worst=d;if(d<tmin)tmin=d;sum+=d;}
  getrusage(RUSAGE_SELF,&b);
  printf("%-18s: faults in loop=%6ld  best=%.2f us  mean=%.2f us  WORST=%.2f
us\n",lock?"with mlock":"without mlock",b.ru_minflt-
a.ru_minflt,tmin,sum/CYCLES,worst);
  return 0;}
```

Compile and run: **gcc -O1 -std=gnu11 q7.c -o q7 && ./q7 0 ; ./q7 1** (three repetitions):

Run	Mode	Page faults in the loop	Mean cycle (µs)	WORST cycle (µs)
1	without mlock	16,387	8.15	126.23
1	with mlock	3	0.19	24.44
2	without mlock	16,387	5.58	73.00
2	with mlock	3	0.17	27.19
3	without mlock	16,387	5.23	70.00
3	with mlock	3	0.11	0.71

Results: without locking, the loop met 16,387 page faults (one for each new page that was touched) and the worst cycle took **70–126 µs**, with the mean 5–8 µs (the page faults make some cycles more than 20 times slower than the average). With locking, the pages are already in RAM, so the loop had practically no faults (3 are from the start of the measuring code) and the mean cycle was 0.11–0.19 µs, the worst cycle between 0.71 and 27 µs. The worst-case delay was reduced by 3 to 100 times, and the mean by about 40 times. The remaining spikes of up to

27 µs in the locked runs are not page faults: they come from other causes (the scheduler and interrupts on a non-real-time Linux kernel on a shared single-core virtual machine); they show that locking removes the page-fault delay but not all jitter, which needs a real-time scheduler (PREEMPT_RT, SCHED_FIFO) and an isolated core.

3.2 Simulation of the memory use of the three tasks

Page-replacement simulator (q7r.c): 1 page = 1 MB; RAM 2,048 pages; the kernel and services keep 256, so 1,792 pages remain. In each step the control task touches its page twice, the ML task scans its 1,536 pages in a cycle (6 passes), logging touches its 128 pages cyclically; a 4th task of 500 pages is added in the second experiment. The pinned (locked) control pages are removed from the pool and never replaced (LRU for the others); the second case, "not pinned", uses FIFO for all the pages:

Tasks	Policy	Control faults	ML faults	Logging faults	4th task faults
3 tasks	Control pinned (mlock) + LRU	0	1,536	128	–
3 tasks	Not pinned, FIFO	64	1,536	128	–
4 tasks (+500 MB)	Control pinned (mlock) + LRU	0	9,216	128	500
4 tasks (+500 MB)	Not pinned, FIFO	512	9,216	1,024	3,216

Verification: with the three tasks all the pages fit into the 1,792 pages (64 + 1,536 + 128 = 1,728), so only the first-use faults occur (1,536 for ML and 128 for logging, i.e. one per page), and no replacement is ever needed. The control task had **0 faults** when its pages were pinned – it is never delayed. If the pages are not pinned and the memory becomes short (4th task), a FIFO policy evicts also the control pages: **512 faults** for the control task – each one a possible deadline miss. So locking is what guarantees R1, not the amount of free memory.

Stage 4 – Defend the Design (2 Marks)

Design: Linux with demand paging for the ML and logging tasks, the control task locked with mlockall() and pre-faulted, real-time priority on a dedicated core, and cgroup memory limits (ML ≤ 1,536 MB, logging ≤ 128 MB). **Justification from the measurements:** (1) locking reduced the page faults in the control loop from 16,387 to 3 and the worst cycle from 70–126 µs to 0.7–27 µs; (2) the simulation shows 0 control faults with locking, while without pinning 512 faults appeared under memory pressure; (3) the budget of 1,984 MB of 2,048 MB meets requirements R3 and R4, and paging has no external fragmentation, so the device can run for a long time while the ML task loads and frees its memory.

Adding a fourth task of 500 MB: the demand becomes 256 + 64 + 1,536 + 128 + 500 = 2,484 MB, which is **436 MB more than the 2,048 MB** available. Because there is no swap, the extra memory cannot be provided by paging out. In the simulation the ML task's faults rose from 1,536 to **9,216** (every reference of the 6 passes faulted: thrashing, because its 1,536 pages no longer fit), although the control task stayed at 0 faults thanks to the locking. The options are: reduce the ML model (quantisation, a smaller model) by about 450 MB, add RAM or a swap/zram area, run the 4th task only when the ML task is idle, or limit it with a cgroup so that it cannot take memory from the others. The locked control memory is not affected in any case.

One remaining limitation: the design guarantees that there are no page faults, but not the full worst-case time: the standard Linux kernel is not a hard real-time kernel, so interrupts, the scheduler, cache and TLB misses and kernel locks still cause jitter (the 27 µs spikes in the locked runs). For a hard guarantee the PREEMPT_RT kernel or a real-time OS and a proof of the worst-case execution time are required. A further limit is that without swap, memory exhaustion leads to the out-of-memory killer.

Alternative concept – an RTOS with no virtual memory

Criterion	Linux with locked memory	RTOS without virtual memory
Worst-case delay of the control task	Page faults removed, but jitter from the kernel remains: measured up to 27 µs on a standard kernel (with PREEMPT_RT, typically tens of µs)	Deterministic and small: no page faults at all, bounded interrupt latency and context switch (a few µs), can be calculated and certified
Memory available to the ML task	About 1,536 MB can be used by ML (the 256 MB kernel and services reserve) and the model can be demand-paged from a file	More physical RAM for the application (the RTOS needs only a few MB) – up to about 1.9 GB – but everything must be resident: no demand paging, no memory-mapped model files, no swap; the ML library may not be available for the RTOS
Effort of adding new tasks	Easy: a new Linux process with a cgroup limit; large software ecosystem; the memory budget must still be checked	Higher: a fixed memory budget for every task decided at design time, recompilation and re-testing of the timing; no protection between tasks unless the CPU has an MPU; fewer libraries

Evaluation: if the control deadline is very strict and certification is required, the RTOS is better for the control task; but because the device also has a 1.5 GB ML task and the vendor wants to add tasks, Linux with locked memory (or a hybrid: an RTOS core for control and Linux for the rest) is the better compromise.

MARK DISTRIBUTION

Component	Task	Bloom's Level	Marks
Part A – Design	A1 – Linux System Components	K5 – Evaluate	7
	A2 – System Administration	K5 – Evaluate	5
	A3 – Virtualisation	K5 – Evaluate	5
	A4 – Containerisation	K5 – Evaluate	4
	A5 – Cloud and Orchestration	K5 – Evaluate	4
	Part A Total		**25**
Part B – Implementation	B1 – Linux Administration	K5 – Evaluate	7
	B2 – Containerisation	K5 – Evaluate	5
	B3 – Deployment Comparison	K5 – Evaluate	8
	B4 – Final Engineering Decision	K5 – Evaluate	5
	Part B Total		**25**
	TOTAL		**50**